36. UPSTREAM OPEN-SOURCE REPOSITORY RESEARCH AND ADOPTION PROGRAM

The project must systematically inspect, benchmark, and selectively adopt ideas from established open-source quantitative finance, investment research, AI-agent, backtesting, data, portfolio-management, and execution repositories.

Do not blindly merge repositories, copy entire codebases, or combine incompatible frameworks.

Popularity, GitHub stars, screenshots, claimed returns, and marketing statements are not sufficient evidence that a repository is suitable for production trading.

Every repository must pass:

- License review
- Architecture review
- Security review
- Dependency review
- Maintenance review
- Testing review
- Data-leakage review
- Backtesting-correctness review
- Live-trading safety review
- Operational-complexity review
- Performance benchmark
- Clean integration assessment

The purpose of this program is to extract proven architectural patterns, research methodologies, workflow designs, data contracts, testing approaches, interface concepts, and reusable components without turning the primary system into an unmaintainable collection of third-party projects.

36.1 REPOSITORY ISOLATION POLICY

Create a separate sibling directory outside the primary application repository:

```
trade-platform/
upstream-research/
```

The `upstream-research` directory must not be imported automatically by the production application.

Preferred structure:

```
upstream-research/
├── README.md
├── REPOSITORY_CATALOG.md
├── LICENSE_MATRIX.md
├── SECURITY_FINDINGS.md
├── ARCHITECTURE_COMPARISON.md
├── FEATURE_COMPARISON.md
├── ADOPTION_DECISIONS.md
├── BENCHMARK_RESULTS.md
├── repositories/
├── extracted-patterns/
├── clean-room-specifications/
└── experiments/
```

Do not commit the complete upstream repositories into the primary product repository.

Use one of the following approaches:

1. Separate shallow clones for research.
2. Git submodules only for approved runtime dependencies.
3. Package-manager dependencies with pinned versions.
4. Clean-room reimplementation based on documented interfaces.
5. External service adapters where license or architecture isolation is necessary.

All upstream code must be considered untrusted until audited.

36.2 INITIAL REPOSITORIES PROVIDED BY THE OPERATOR

Clone and inspect the following repositories:

```
mkdir -p ../upstream-research/repositories
cd ../upstream-research/repositories

git clone --depth 1 --filter=blob:none
  https://github.com/coding-kitties/investing-algorithm-framework.git

git clone --depth 1 --filter=blob:none
  https://github.com/kernc/backtesting.py.git

git clone --depth 1 --filter=blob:none
  https://github.com/tradermonty/claude-trading-skills.git

git clone --depth 1 --filter=blob:none
  https://github.com/TauricResearch/TradingAgents.git

git clone --depth 1 --filter=blob:none
  https://github.com/HKUDS/Vibe-Trading.git
```

```
git clone --depth 1 --filter=blob:none
  https://github.com/Fincept-Corporation/FinceptTerminal.git

git clone --depth 1 --filter=blob:none
  https://github.com/stefan-jansen/machine-learning-for-trading.git

git clone --depth 1 --filter=blob:none
  https://github.com/xbtlin/ai-berkshire.git
```

For very large repositories, use sparse checkout when only specific directories are needed.

Do not execute installation scripts, Docker Compose files, setup scripts, notebook cells, downloaded binaries, or unknown shell commands before completing a security review.

36.3 ADDITIONAL HIGH-VALUE REPOSITORIES

Also inspect:

```
git clone --depth 1 --filter=blob:none
  https://github.com/QuantConnect/Lean.git

git clone --depth 1 --filter=blob:none
  https://github.com/nautechsystems/nautilus_trader.git

git clone --depth 1 --filter=blob:none
  https://github.com/microsoft/qlib.git

git clone --depth 1 --filter=blob:none
  https://github.com/AI4Finance-Foundation/FinRL-Trading.git

git clone --depth 1 --filter=blob:none
  https://github.com/OpenBB-finance/OpenBB.git

git clone --depth 1 --filter=blob:none
  https://github.com/freqtrade/freqtrade.git

git clone --depth 1 --filter=blob:none
  https://github.com/polakowo/vectorbt.git

git clone --depth 1 --filter=blob:none
  https://github.com/AI4Finance-Foundation/FinRobot.git
```

These repositories are reference candidates, not automatically approved dependencies.

36.4 INTENDED ROLE OF EACH REPOSITORY

Investing Algorithm Framework

Investigate for:

- Strategy interface design
- Vectorized research stage
- Event-driven validation stage
- Strategy comparison reports
- Performance metric calculation
- Strategy ranking
- Research-to-deployment workflow
- Experiment result visualization

Potential adoption:

- Research experiment interface
- Strategy comparison schema
- Metric definitions
- HTML report concepts
- Two-stage backtest workflow

Do not assume its deployment layer satisfies the project's broker, reconciliation, risk, and audit requirements.

Backtesting.py

Investigate for:

- Minimal strategy API
- Fast baseline backtests
- Indicator registration
- Parameter optimization
- Trade and position abstractions
- Interactive result visualization
- Simple educational strategies

Potential adoption:

- Baseline backtest adapter
- Unit-test reference engine
- Fast prototype mode
- Cross-engine validation

Do not use it as the sole production event simulator.

Its results must be compared against the primary event-driven engine.

Claude Trading Skills

Investigate for:

- Skill manifests
- Canonical skill indexing
- Workflow manifests
- Position-sizing routines
- Market-regime reviews
- Technical-analysis workflows
- Trade journals
- Signal postmortems
- Portfolio-review workflows
- Human approval gates
- Skill-quality scoring
- Continuous workflow improvement

Potential adoption:

- Internal AI skill registry
- Research workflow manifests
- Trade-review templates
- Postmortem processes
- Decision-journal structures
- Skill testing and validation framework

Do not use an LLM skill as a substitute for deterministic financial calculation or the Risk Engine.

TradingAgents

Investigate for:

- Multi-agent graph architecture
- Specialized analyst roles
- Fundamental analyst
- Technical analyst
- Sentiment analyst
- News analyst
- Bull researcher
- Bear researcher
- Trader agent
- Risk-management agents
- Portfolio manager
- Structured outputs
- Persistent decision state
- Agent debates
- Checkpoint and resume behavior
- Multi-model provider abstraction

Potential adoption:

- Research-agent orchestration

- Bull versus bear debate
- Independent risk challenge
- Decision synthesis
- Structured agent outputs
- Persistent research sessions

Do not allow LLM agents to submit orders directly.

All agent outputs must become structured evidence objects processed by deterministic signal, portfolio, and risk services.

Vibe-Trading

Investigate for:

- Natural-language research interface
- Research memory
- Multi-agent research teams
- Market-data loaders
- Multi-market support
- Backtest adapters
- Alpha-factor catalog
- Alpha benchmarking
- MCP integration
- Reusable finance skills
- Strategy export
- Run cards
- Research artifacts
- Shadow-account concepts
- Web and CLI interfaces

Potential adoption:

- Natural-language research workspace
- Agent tool registry
- Research memory model
- Alpha-zoo catalog format
- Backtest run-card format
- MCP server architecture
- Research artifact generation
- Shadow portfolio comparison

Do not treat generated strategy code as trusted.

Generated code must be sandboxed, statically analyzed, tested, and prohibited from reaching live execution without formal promotion.

Fincept Terminal

Investigate for:

- Terminal information architecture

- Screen hierarchy
- Multi-asset navigation
- Equity research interface
- Portfolio interface
- News interface
- Economic-data interface
- Node-based workflows
- Agent catalog
- Data-connector catalog
- Broker capability matrix
- Quant analytics presentation
- Desktop performance concepts

Potential adoption:

- Product requirements
- Screen inventory
- UX inspiration
- Data-provider categories
- Broker research
- Financial analysis feature inventory

Do not copy source code, branding, icons, screenshots, layouts, or protected assets without explicit license approval.

Because its technology stack differs from the proposed web platform, treat it primarily as a feature and UX reference.

Machine Learning for Trading

Investigate for:

- Point-in-time data practices
- Market and fundamental data sourcing
- Alternative data
- Alpha-factor research
- Feature engineering
- ML workflow design
- Model validation
- NLP for filings and news
- Time-series models
- Portfolio optimization
- Strategy evaluation
- Transaction-cost modeling
- Reinforcement-learning examples

Potential adoption:

- Research methodology
- Reproducible notebooks converted into tested modules
- Feature definitions
- Model baselines

- Evaluation protocols
- Educational documentation

Do not copy notebook code directly into production.

Every adopted notebook concept must be rewritten as:

- A versioned module
- A tested pipeline
- A documented feature
- A reproducible experiment
- A leakage-safe implementation

AI Berkshire

Investigate for:

- Value-investing research workflows
- Multi-perspective investment analysis
- Business-quality analysis
- Moat analysis
- Management analysis
- Valuation
- Margin-of-safety logic
- Earnings-review workflows
- Industry funnels
- Investment checklists
- Thesis tracking
- Thesis-drift detection
- Contrarian analysis
- Inversion analysis
- Information-richness scoring
- Data cross-validation
- Financial-calculation verification

Potential adoption:

- Long-term investment research skills
- Investment thesis schema
- Anti-bias checklist
- Source cross-validation requirements
- Bear-case challenge process
- Thesis invalidation monitoring
- Structured company comparison

Do not accept repository performance claims as evidence of strategy quality.

Do not emulate named investors as unquestionable authorities.

Convert every perspective into explicit, testable evaluation dimensions.

QuantConnect Lean

Investigate for:

- Professional event-driven architecture
- Security master
- Multi-asset modeling
- Corporate actions
- Order lifecycle
- Brokerage adapters
- Data subscriptions
- Universe selection
- Scheduling
- Cash book
- Currency conversion
- Margin models
- Fee models
- Slippage models
- Live/backtest parity
- Algorithm framework components

Potential adoption:

- Production architecture reference
- Broker abstraction concepts
- Instrument and order domain models
- Event-loop design
- Corporate-action handling
- Security and portfolio accounting concepts

Evaluate whether Lean should become:

- A separate validation engine
- A specialist execution engine
- A supported strategy export target
- An external worker service

Do not integrate it merely because it is comprehensive. Compare operational cost and stack complexity against the Python/Rust architecture.

NautilusTrader

Investigate for:

- Deterministic event-driven simulation
- Nanosecond time model
- Multi-venue architecture
- Research-to-live parity
- Order book simulation
- Advanced order types
- Execution state machine
- Instrument definitions

- Message bus
- Cache
- Broker and exchange adapters
- Rust performance core
- Python control plane
- State persistence
- Reconciliation patterns

Potential adoption:

- Primary event-driven execution engine
- Execution reference implementation
- Live-trading adapter layer
- High-fidelity simulation
- Order and position domain model
- Multi-venue orchestration

Perform a dedicated proof of concept comparing NautilusTrader with an internally developed execution engine before selecting either approach.

Microsoft Qlib

Investigate for:

- Data layer
- Dataset handlers
- Feature storage
- Model workflows
- Experiment management
- Forecast models
- Portfolio strategy
- Online learning
- Workflow orchestration
- AI-based quantitative research

Potential adoption:

- ML research subsystem
- Feature and dataset interfaces
- Model experiment workflows
- Research benchmark environment

Do not couple live execution directly to Qlib research objects.

FinRL-X

Investigate for:

- Weight-centric strategy contracts
- Stock-selection modules
- Portfolio-allocation modules
- Timing overlays

- Risk overlays
- Deployment consistency
- Paper and live execution
- Multi-account support
- Broker integration
- ML and reinforcement-learning pipelines

Potential adoption:

- Canonical target-weight interface
- Strategy composition
- Portfolio rebalance contracts
- Research-to-execution consistency
- Risk overlay interface

The target-weight contract is a strong candidate for the internal boundary between strategy research and portfolio construction.

OpenBB

Investigate for:

- Data-provider abstraction
- Provider extension system
- Financial-data normalization
- FastAPI exposure
- Analyst and AI-agent data access
- Financial command taxonomy
- Data-source fallback

Potential adoption:

- Data-provider adapter patterns
- Research data API
- Connector registry
- Provider capability metadata

Review AGPL obligations before using code in the main platform.

Freqtrade

Investigate for:

- Crypto exchange adapters
- Dry-run mode
- Backtesting
- Hyperparameter optimization
- Look-ahead analysis
- Recursive feature analysis
- Dynamic pair lists
- Strategy isolation
- Operational commands

- Web management
- Telegram notifications
- FreqAI
- Exchange-specific configuration

Potential adoption:

- Crypto-specific operational patterns
- Dry-run safety
- Exchange capability metadata
- Strategy validation tools
- Look-ahead detection
- Bot lifecycle management

Do not make the entire platform crypto-specific.

VectorBT

Investigate for:

- Massive parameter sweeps
- Multi-asset broadcasting
- Vectorized portfolio simulation
- Signal arrays
- Fast indicator calculation
- Walk-forward testing
- Parameter heatmaps
- Large-scale factor research
- Numba and Rust acceleration

Potential adoption:

- High-throughput research engine
- Parameter-screening stage
- Feature benchmarking
- Strategy candidate generation

Use it only after license approval.

Vectorized results must be validated through an event-driven engine before strategy promotion.

FinRobot

Investigate for:

- Financial agent workflows
- Annual-report analysis
- Equity-research reports
- Valuation agents
- Risk agents
- Financial document processing
- Multi-agent report production

Potential adoption:

- Research report generation
- Filing-analysis workflows
- Valuation-agent prompts
- Research artifact templates

Do not use generated reports as an order trigger without structured evidence validation.

36.5 LICENSE GATE

Create `LICENSE_MATRIX.md` with the following fields:

```
Repository
License
Copyright holder
Commercial use allowed
Modification allowed
Distribution obligations
Network-use obligations
Patent grant
Trademark restrictions
Attribution requirements
Source-disclosure risk
Compatible with private use
Compatible with future SaaS
Compatible with proprietary core
Legal review required
Approved use category
```

Approved use categories:

```
RUNTIME_DEPENDENCY
OPTIONAL_ADAPTER
SEPARATE_SERVICE
RESEARCH_ONLY
DESIGN_REFERENCE
CLEAN_ROOM_REIMPLEMENTATION
REJECTED
LEGAL_REVIEW_REQUIRED
```

No repository may become a runtime dependency before its license category is documented.

Special attention is required for:

- AGPL
- GPL

- LGPL
- Commons Clause
- Dual licenses
- Commercial-use restrictions
- Trademark restrictions
- Data-provider terms
- Model-weight licenses
- Included datasets
- Included images and UI assets

Future commercialization must be considered even though the first version is private.

36.6 SECURITY AND SUPPLY-CHAIN AUDIT

Before executing any repository:

1. Inspect installation scripts.
2. Inspect shell scripts.
3. Inspect Dockerfiles.
4. Inspect Docker Compose files.
5. Inspect GitHub Actions.
6. Inspect package manifests.
7. Inspect post-install hooks.
8. Inspect downloaded binaries.
9. Inspect external URLs.
10. Inspect telemetry.
11. Inspect credential handling.
12. Inspect file-system access.
13. Inspect subprocess execution.
14. Inspect dynamic code execution.
15. Inspect deserialization.
16. Inspect notebook outputs.
17. Inspect model downloads.
18. Inspect MCP tools.
19. Inspect browser automation.
20. Inspect network listeners.

Run:

- Secret scanning
- Dependency vulnerability scanning
- Static application security testing
- Container scanning
- License scanning
- Software bill of materials generation
- Malicious-package checks
- Typosquatting checks

All experiments must run inside isolated containers with:

- No broker credentials
- No production secrets
- No unrestricted host mounts
- No live-trading access
- No cloud-admin credentials
- Restricted outbound network access where possible
- Resource limits
- Read-only source mounts where possible

36.7 ARCHITECTURE COMPARISON

Create `ARCHITECTURE_COMPARISON.md`.

Compare every repository across:

- Primary purpose
- Main language
- Architecture style
- Backtesting model
- Event-driven support
- Vectorized support
- Multi-asset support
- Multi-venue support
- Tick support
- Order-book support
- Corporate actions
- Portfolio accounting
- Risk engine
- Broker integrations
- Live execution
- Paper trading
- Data providers
- Fundamental analysis
- News analysis
- Sentiment analysis
- ML support
- LLM-agent support
- Model registry
- Experiment tracking
- Audit logging
- Reconciliation
- UI
- API
- MCP
- Testing maturity
- Deployment maturity
- Extensibility

- Documentation
- License
- Operational complexity

Do not select a winner using one aggregate score alone.

Produce separate recommendations for:

- Research engine
- Event-driven simulator
- Live execution engine
- Data platform
- AI-agent orchestration
- Long-term investment research
- Portfolio construction
- Crypto execution
- Dashboard design

36.8 ADOPTION DECISION PROCESS

For every reusable component, create an Adoption Decision Record.

Required format:

```
Decision ID:
Source repository:
Source version or commit:
Component or pattern:
Problem solved:
Alternatives considered:
License status:
Security status:
Performance status:
Testing status:
Integration complexity:
Operational cost:
Vendor-lock-in risk:
Decision:
Adoption method:
Files affected:
Required attribution:
Rollback plan:
Owner:
Review date:
```

Valid decisions:

```
ADOPT
ADAPT
WRAP
REIMPLEMENT
REFERENCE_ONLY
REJECT
DEFER
```

Definitions:

- `ADOPT`: Use the dependency with minimal changes.
- `ADAPT`: Fork or modify after license approval.
- `WRAP`: Keep isolated behind an internal adapter.
- `REIMPLEMENT`: Create a clean internal implementation based on behavior and public interfaces.
- `REFERENCE_ONLY`: Use only for learning or design inspiration.
- `REJECT`: Do not use.
- `DEFER`: Revisit later.

36.9 PROPOSED COMPOSITE ARCHITECTURE

The preferred architecture should combine ideas, not entire repositories.

Quantitative Research Plane

Use concepts from:

- Investing Algorithm Framework
- VectorBT
- Backtesting.py
- Qlib
- Machine Learning for Trading
- FinRL-X

Responsibilities:

- Feature research
- Parameter sweeps
- Alpha discovery
- Baseline backtests
- ML experiments
- Portfolio-weight generation
- Strategy comparison

High-Fidelity Simulation and Execution Plane

Evaluate concepts or components from:

- NautilusTrader
- QuantConnect Lean
- Freqtrade for crypto-specific operations

Responsibilities:

- Event-driven simulation
- Order lifecycle
- Partial fills
- Fees
- Slippage
- Funding
- Margin
- Broker adapters
- Exchange adapters
- Reconciliation
- Live execution

AI Research and Decision-Support Plane

Use patterns from:

- TradingAgents
- Vibe-Trading
- Claude Trading Skills
- FinRobot
- AI Berkshire

Responsibilities:

- Fundamental research
- Technical research
- Macro research
- News and sentiment analysis
- Bull/bear debate
- Thesis generation
- Thesis challenge
- Investment reporting
- Trade journaling
- Postmortem
- Natural-language queries

This plane may propose evidence and recommendations but must not directly control execution.

Data and Intelligence Plane

Use patterns from:

- OpenBB
- Fincept Terminal
- Qlib
- Vibe-Trading

Responsibilities:

- Provider adapters
- Data normalization
- Provider fallback
- Data provenance
- Financial-data APIs
- Connector registry
- Data-health monitoring

Operator Experience Plane

Use product inspiration from:

- Fincept Terminal
- Vibe-Trading
- OpenBB Workspace concepts
- Investing Algorithm Framework reports

Responsibilities:

- Command center
- Market intelligence
- Instrument workstation
- Strategy laboratory
- Risk center
- Investment center
- Agent research center
- Audit center
- Data-health center

36.10 MANDATORY CROSS-ENGINE VALIDATION

No strategy may be promoted because it performs well in one backtesting engine.

For every candidate strategy:

1. Run the fast vectorized backtest.
2. Run a second independent baseline engine.
3. Run the primary event-driven engine.
4. Compare orders and fills.

5. Compare fees.
6. Compare slippage.
7. Compare corporate-action treatment.
8. Compare position accounting.
9. Compare timestamps.
10. Explain every material difference.

Create automated cross-engine reconciliation reports.

A strategy must be blocked when:

- Results diverge materially without explanation.
- Trade timestamps differ unexpectedly.
- Look-ahead behavior is detected.
- Position accounting differs.
- Fees are omitted.
- Unrealistic fills are required.
- Event-driven results invalidate the vectorized edge.

36.11 ANTI-PATTERNS

Do not:

- Combine every framework into one runtime.
- Let LLM agents place orders directly.
- Trust self-reported returns.
- Select tools based only on stars.
- Copy UI assets.
- Ignore licenses because the project is currently private.
- Execute unknown setup scripts on the host machine.
- Use notebook code directly in production.
- Make a research framework responsible for broker reconciliation.
- Use a vectorized backtester as the only execution simulator.
- Use an LLM-generated strategy without leakage tests.
- Treat sentiment as a standalone signal.
- enable live trading during repository evaluation.
- store upstream API keys in the research clones.
- expose the research environment publicly.
- allow upstream dependencies to update without version pinning.

36.12 FIRST UPSTREAM RESEARCH ASSIGNMENT

Perform the following work before selecting the final implementation stack:

1. Clone the repositories into the isolated research directory.
2. Record the exact commit SHA for every repository.
3. Generate a software bill of materials.
4. Record each license.

5. Inspect repository activity and release history.
6. Map the directory structure.
7. Identify core abstractions.
8. Identify external services.
9. Identify broker and data-provider integrations.
10. Identify test coverage.
11. Identify security risks.
12. Identify code-execution risks.
13. Identify data-leakage risks.
14. Identify reusable patterns.
15. Identify incompatible architectural assumptions.
16. Benchmark installation complexity.
17. Benchmark one equivalent moving-average strategy.
18. Benchmark one portfolio strategy.
19. Compare result metrics.
20. Produce adoption decisions.

Create:

```
/docs/upstream/REPOSITORY_CATALOG.md
/docs/upstream/LICENSE_MATRIX.md
/docs/upstream/SECURITY_FINDINGS.md
/docs/upstream/ARCHITECTURE_COMPARISON.md
/docs/upstream/FEATURE_COMPARISON.md
/docs/upstream/BACKTEST_ENGINE_COMPARISON.md
/docs/upstream/AGENT_ARCHITECTURE_COMPARISON.md
/docs/upstream/DATA_CONNECTOR_COMPARISON.md
/docs/upstream/ADOPTION_DECISIONS.md
/docs/upstream/BENCHMARK_RESULTS.md
/docs/upstream/ATTRIBUTIONS.md
```

The final report must explicitly state:

- Which repositories will become dependencies
- Which will remain isolated services
- Which will only provide architectural inspiration
- Which require legal review
- Which are rejected
- Why each decision was made
- What original code will be built internally
- How future upstream updates will be monitored

36.13 INITIAL RECOMMENDED ADOPTION HYPOTHESIS

Treat the following as an initial hypothesis to validate, not a final decision.

Strong Direct References

- Vibe-Trading

- TradingAgents
- Investing Algorithm Framework
- Machine Learning for Trading
- AI Berkshire
- FinRL-X
- QuantConnect Lean
- NautilusTrader

Optional Isolated Adapters

- Backtesting.py
- Qlib
- Freqtrade
- OpenBB
- VectorBT
- FinRobot

Primarily Product and UX Reference

- Fincept Terminal

Initial Architecture Hypothesis

- Internal Python services for data, research, risk, portfolio, API, and AI orchestration
- Rust-backed or mature external engine for high-fidelity event simulation and eventual execution
- Vectorized engine for high-throughput candidate screening
- Independent Risk Engine owned by the primary platform
- Broker-neutral OMS and reconciliation layer
- LLM agents restricted to research and structured recommendations
- Separate active-trading and long-term-investment domains
- Versioned strategy, feature, dataset, model, prompt, and decision registries

Do not finalize this hypothesis until repository audits and proof-of-concept benchmarks are complete.

36.14 ACCEPTANCE CRITERIA

The upstream research phase is complete only when:

- Every repository has an exact version record.
- Every repository has a license classification.
- Every executable repository has a security assessment.
- Every candidate dependency has a software bill of materials.
- Major backtest engines have been compared using identical data.
- Material result differences have been explained.
- Agent frameworks have been evaluated for determinism and auditability.
- No LLM agent can bypass deterministic risk controls.
- No restricted-license code has entered the proprietary core.
- Every adoption decision is documented.
- Production architecture does not depend on unverified performance claims.
- The primary system remains operable if any optional AI-agent framework is removed.
- The Risk Engine and execution safety layer remain controlled by the primary platform.